

Datawarehouse Implementation



Na vandaag...

Weet/ken je...

Kan je...

Welke soorten databronnen er bestaan

Wat Slowly Changing Dimensions typen 0 t/m 3 zijn, hoe zij van elkaar verschillen en waarom/wanneer je welk type gebruikt.

Wat in Datawarehouses het verschil is tussen Business Keys & Surrogate Keys

Welke inlaadstrategieën je het beste kunt toepassen bij het inladen van data in Datawarehouses

Slowly Changing Dimensions typen 1 en 2 inbouwen in dynamische Datawarehouses

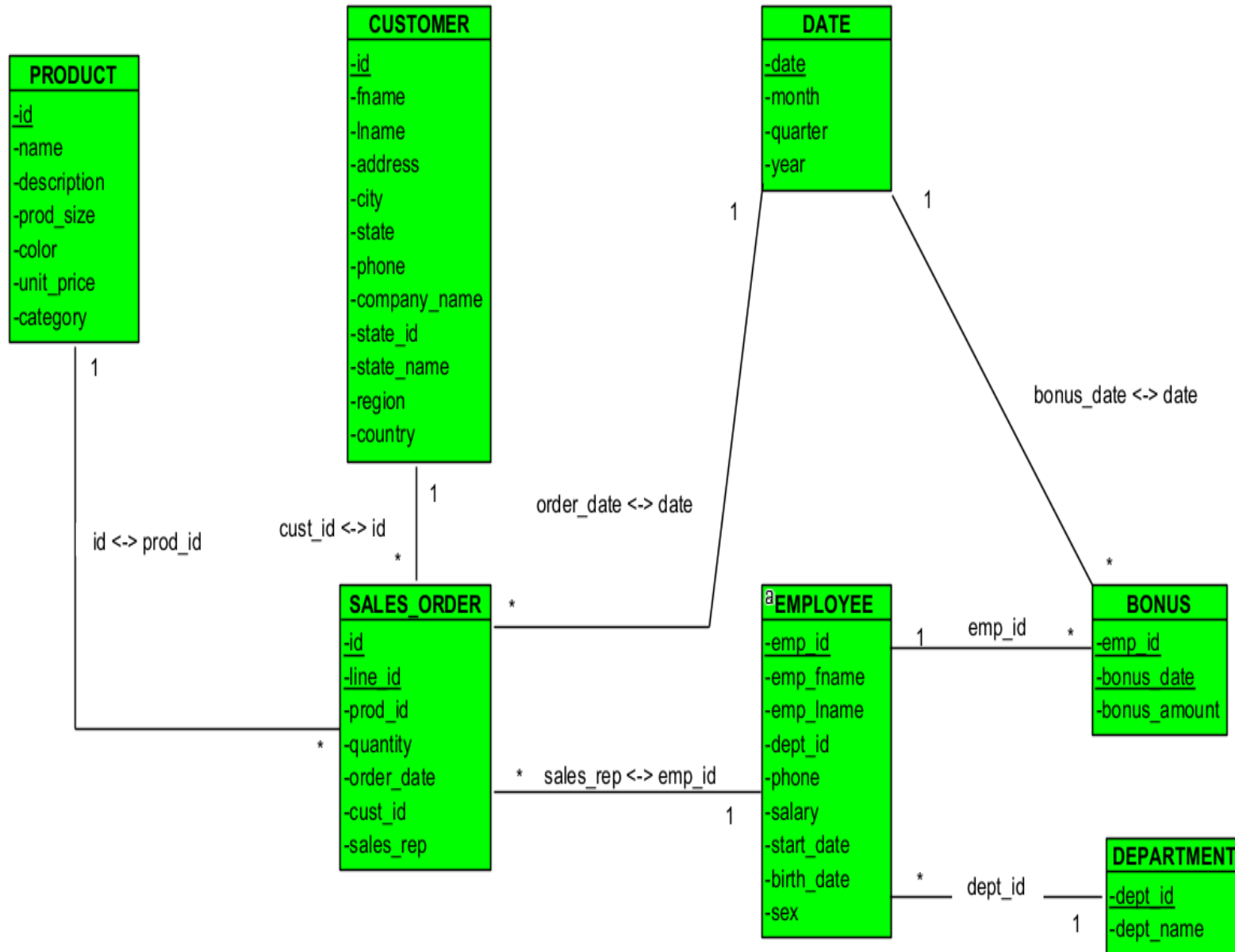
Surrogate- & Business Keys op de juiste manier verwerken in Datawarehouses

De strategieën "UPSERT" en "Incremental Data Loading" gebruiken om Datawarehouses te actualiseren met data uit Source Data Modellen

HC + WC



Het huidige Sterschema

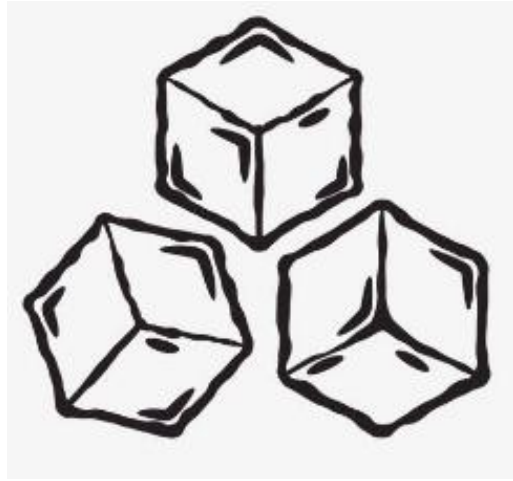


Bij het aanmaken van het DWH: maak je als eerst de feiten of juist als eerst de dimensies aan?

De dimensies: feiten moeten hiernaar verwijzen, dus deze moeten al bestaan

2 soorten databronnen

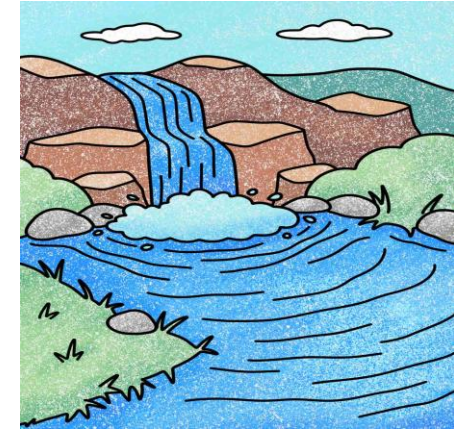
Statisch



Databronnen waarin bijna nooit wijzigingen plaatsvinden. Bijvoorbeeld...

- Overzichten die slechts eens per jaar geraadpleegd worden
- Overzichten die in nieuwe bestanden zijn geactualiseerd
- Databases van softwaresystemen die niet (meer) gebruikt worden

Dynamisch



Databronnen waarin (heel) vaak wijzigingen plaatsvinden. Bijvoorbeeld...

- Databases waarin updates plaatsvinden bij elke klantactie.
- “Getapte” / “gescrapete” data van continu wijzigende webpagina’s.
- Platformen waarop continu wijzigende sensordata terechtkomt (Internet of Things).

Datawarehouses en het bijhouden van historie

Dimensie: Klant

<u>Nummer_PK</u>	Naam	Woonplaats
------------------	------	------------

Verhuisd op 01-01-2026

Bij deze historische verkoop “denkt” het DWH dat het gaat om een klant uit Den Haag, terwijl hij toen nog in Lelystad woonde!

Dit kan een integriteitsfout zijn: een geschiedenis is overschreven met de actualiteit, waardoor historische waarden minder accuraat worden weergegeven.

Stel: we meten (op een hoger abstractieniveau) de totaalprijs per woonplaats.

Dan...

- Wordt in de nieuwe stad (Den Haag) 30 euro **hoger** gemeten dan de werkelijkheid.
- Wordt in de oude stad (Lelystad) 30 euro **lager** gemeten dan de werkelijkheid.

Dit hoeft geen probleem te zijn, als je je er maar bewust van bent.

Hier kan je per dimensie op 4 verschillende manieren mee omgaan: Slowly Changing Dimensions.

Feit: Bestelling

<u>Bestelling_PK</u>	Datum	Totaal-prijs	Klant_FK
1	31-12-2025	30	1

Slowly Changing Dimension (SCD) Type 0

Provinces of
the Netherlands



Operationele data

<u>Nummer</u> PK	Naam	Woonplaats
------------------	------	------------

UPDATE Klant (...)

Datawarehouse-data

<u>Nummer</u> PK	Naam	Woonplaats
1	Henk	Lelystad

Bij SCD-type 0 blijft de oude waarde in het DWH staan, deze wordt dan dus niet overschreven met de nieuwe waarde.

Niet vaak gebruikt, omdat je voor de beste bedrijfsbeslissingen de meest actuele data wilt hebben.

Slowly Changing Dimension (SCD) Type 1

Provinces of
the Netherlands



Operationele data

<u>Nummer</u> PK	Naam	Woonplaats
------------------	------	------------

UPDATE Klant (...)

Datawarehouse-data

<u>Nummer</u> PK	Naam	Woonplaats
------------------	------	------------

Bij SCD-type 1 wordt de oude waarde in het DWH volledig overschreven door de nieuwe waarde.

Vaak gebruikt, opdrachtgevers accepteren kleine historische onnauwkeurigheden.

Slowly Changing Dimension (SCD) Type 2

Provinces of
the Netherlands



Operationele data

<u>Nummer</u> PK	Naam	Woonplaats
------------------	------	------------

UPDATE Klant (...)

Datawarehouse-data

<u>Nummer</u> PK	Naam	Woonplaats
1	Henk	Lelystad

Bij SCD-type 2 komt er per wijziging in de brondata (UPDATE) een nieuwe rij bij in het Datawarehouse (INSERT)

Vaak gebruikt, dit type biedt namelijk het meest volledige en accurate beeld van de bedrijfshistorie.

Slowly Changing Dimension (SCD) Type 3

Provinces of
the Netherlands



Operationele data

<u>Nummer</u> PK	Naam	Woonplaats
------------------	------	------------

UPDATE Klant (...)

Datawarehouse-data

<u>Nummer</u> PK	Naam	Woonplaats	VorigeWoon plaats
------------------	------	------------	----------------------

Bij SCD-type 3 wordt per wijziging in de brondata de “vorige”-kolom gevuld met de oude waarden.

Niet vaak gebruikt, vereist heel veel kolommen (per kolom een “vorige”) voor slechts een beperkt beeld op de historie.

Benodigde extra technieken bij SCD-type 2

Provinces of the Netherlands



Operationele data

<u>Nummer</u> PK	Naam	Woonplaats
1	Henk	Den Haag

Datawarehouse-data

<u>Nummer</u> PK	Naam	Woonplaats
1	Henk	Lelystad
1	Henk	Den Haag

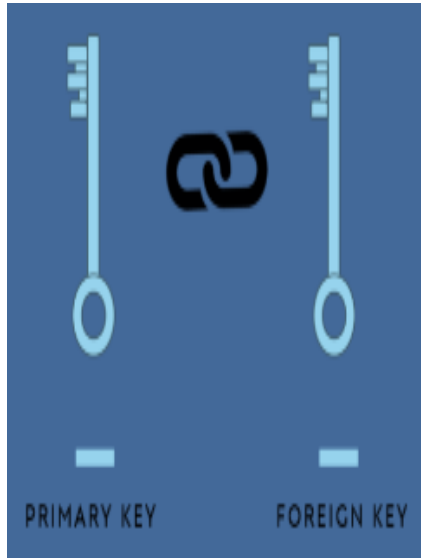
Dit is technisch niet mogelijk, elke PK-waarde moet namelijk uniek zijn!

Hoe op te lossen?

Met een extra primaire sleutel die niet uit de operationele data komt, de zogeheten surrogaatsleutel.

2 soorten identificerende sleutels in het Datawarehouse

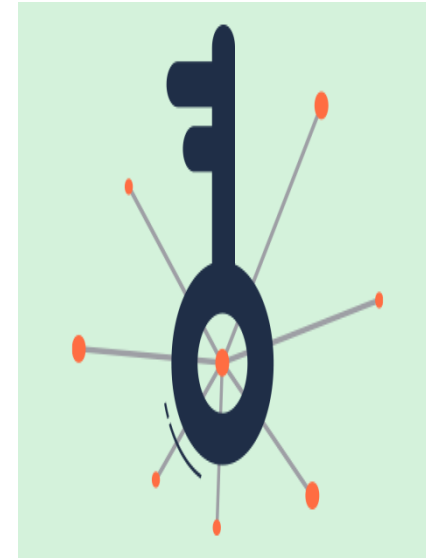
Business Keys op operationeel niveau



- Hetzelfde als primary keys in OLTP-databases
- Kunnen samengesteld zijn
- Bestaan uit (combinaties van) nummers, codes en ID's
- Hiernaar wordt verwezen door **Foreign Keys op operationeel niveau**
- Nummer_PK

Surrogaatsleutels op Datawarehouse-niveau

- Zijn niet samengesteld
- Bestaan slechts uit nummers
- Hiernaar wordt verwezen door **Foreign Keys op Datawarehouse-niveau**
- Nummer_SK



Elke regel in een SCD-type 2 staat voor elke keer dat er in een operationele rij minstens één datamutatie heeft plaatsgevonden.

Het betreffende begin- en eindtijdstip dient óók bijgehouden te worden!

<u>Nummer_SK</u>	Begintijd	Eindtijd	Nummer_PK	Naam	Woonplaats
1	01-01-2025 00:00 AM	01-01-2026 12:00 PM	1	Henk	Lelystad
2	01-01-2026 12:00 PM	NULL	1	Henk	Den Haag



Eindtijd IS NULL, actuele regel!

Per Sterschema-tabel een tabel aanmaken in SQLite

Tabelstructuren creëren

```
CREATE TABLE Sales_Order_Item  
(  
  id INTEGER,  
  line_id INTEGER,  
  prod_id INTEGER,  
  quantity INTEGER,  
  order_date DATE,  
  (...)  
  unit_prize FLOAT,  
  turnover FLOAT,  
  
  PRIMARY KEY(id, line_id),  
  FOREIGN KEY(prod_id)  
  REFERENCES  
  product(product_sk)  
);
```

Let erop dat elke foreign key van een feit verwijst naar de surrogate key van een dimensie.

Tabelstructuren bewerken

Quantity vergeten toe te voegen?

```
ALTER TABLE  
Sales_Order_Item  
ADD quantity INTEGER;
```

Toewijzing primaire sleutel vergeten?

```
ALTER TABLE  
Sales_Order_Item  
ADD PRIMARY  
KEY(id, line_id);
```

Tabelstructuren verwijderen

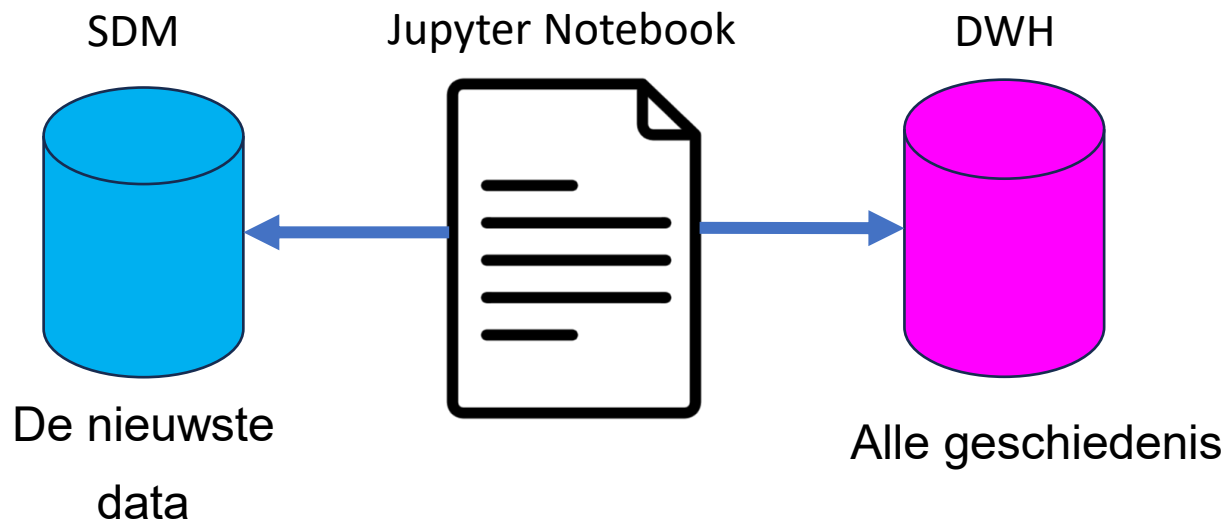
Één kolom verwijderen?

```
ALTER TABLE  
Sales_Order_Item  
DROP COLUMN nutteloos;
```

Hele tabel verwijderen?

```
DROP TABLE Sales_Order_Item;
```

Nieuwe connecties leggen



```
conn_sdm = get_conn(SDM_PATH)
conn_dwh = get_conn(DWH_PATH)
```

Vorige keer consulteerden we meerdere operationele databases met `pandas.read_sql(...)`

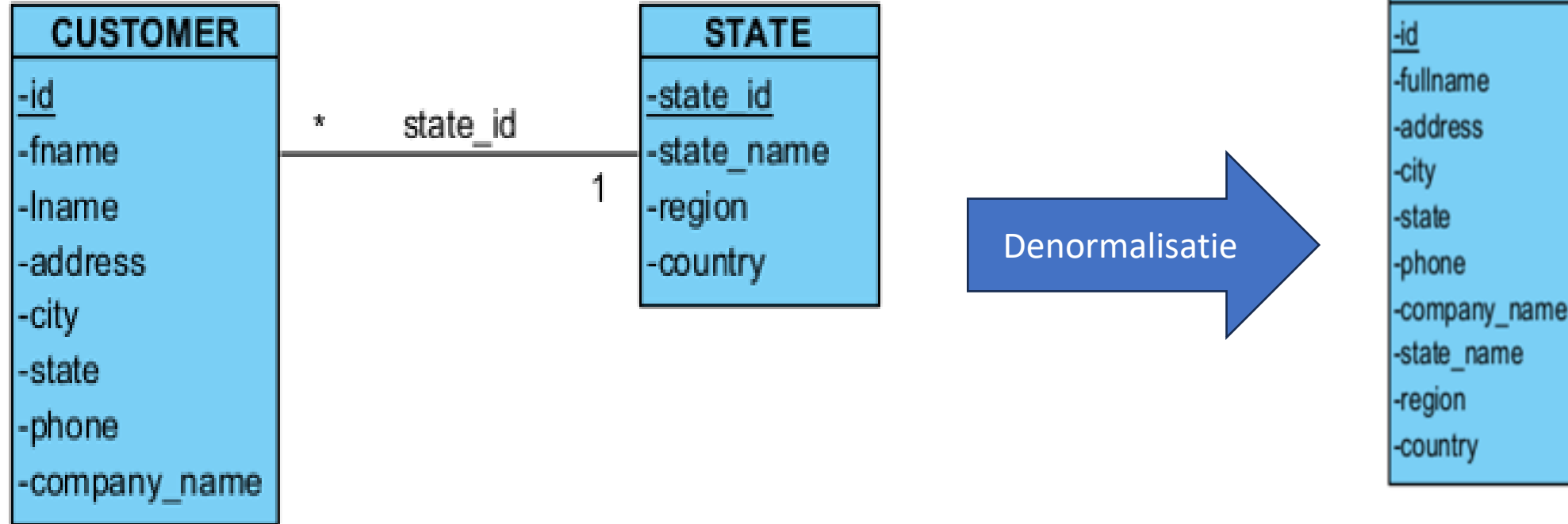
Nu hoeven we slechts 2 connecties te leggen:

- 1 met het SDM, waar de in te laden data vandaan komt.
- 1 met het Datawarehouse, waar de in te laden data naartoe moet, volgens de stappen uit de gemodelleerde ETL-schema's.

Van operationele- naar dimensietabellen d.m.v. merging

Genormaliseerde operationele tabellen

Gedenormaliseerde dimensietabel



Data is eerst genormaliseerd: verspreid over meerdere tabellen.

Maar... bij Datawarehouse Engineering gaan we denormaliseren.

We voegen data dus samen in één tabel, dit moet ook blijken uit je ETL-schema's ("JOIN").

Inlaadstrategieën en Slowly Changing Dimensions

	String-built SQL	Schema-driven Bulk Loading
Full Refresh	- Eenvoud: makkelijk te debuggen - Snelheid: heel langzaam. Onveranderde data wordt telkens opnieuw ingeladen én dit gebeurt rij voor rij.	- Eenvoud: makkelijk te debuggen - Snelheid: langzaam. Onveranderde data wordt nog steeds (overbodig opnieuw) ingeladen, maar nu met 1 SQL-query per batch, i.p.v. per individuele rij.
Incremental Data Loading	- Eenvoud: moeilijk te debuggen. Je ziet lastiger welke individuele rijen misgaan. - Snelheid: snel, omdat er alléén naar gewijzigde data gekeken wordt.	- Eenvoud: moeilijk te debuggen. Je ziet lastiger welke individuele rijen misgaan. - Snelheid: heel snel, omdat er alléén naar gewijzigde data gekeken wordt én er 1 SQL-query per batch is, i.p.v. per individuele rij.

- Voor SCD-type 1 kan je “Full Refresh” én “Incremental Data Loading” gebruiken: met beide methoden kan je namelijk de meest actuele data overhouden.
- Voor SCD-type 2 kan je alleen “Incremental Data Loading” gebruiken, omdat je bij “Full Refresh” altijd je hele geschiedenis weggooit, terwijl je die juist analyseerbaar wil maken.
- Voor de keuze tussen “String-built SQL” of “Schema-driven Bulk Loading” maakt het SCD-type niet uit.

SCD-type 2 uitvoeren met de UPSERT-strategie

“merge” elke tabel uit het SDM met zijn tegenhanger in het DWH.

SDM (de nieuwste data)

PK	Naam	Woonplaats
1	Piet	Leimuiden
2	Elske	Leiden
3	Jan	Lisse

DWH (alle geschiedenis)

SK	Begintijd	Eindtijd	PK	Naam	Woonplaats
1	2025-01-01	NULL	1	Piet	Leiden
2	2025-01-01	NULL	2	Elske	Leiden
3	2025-01-01	NULL	3	Jan	Lisse

Kennelijk is Piet verhuisd van Leiden naar Leimuiden!

Je Pythonscript moet bij elke SCD-type 2 dergelijke veranderingen detecteren en afhandelen met een UPSERT-strategie: per geüpdate rij in het SDM een INSERT in het DWH.

Loop alle rijen uit het SDM langs en vergelijk deze met de DWH-inhoud

- Is de rij uit het SDM anders of bestaat deze nog niet in het DWH? Extra insert
- Anders: niks, ga naar volgende rij.

Rijen die niet meer bestaan, bijvoorbeeld voormalige klanten, moeten wél in het DWH blijven staan (met bekende eindtijd), maar niet (meer) in het SDM.

De geschiedenis moet intact en volledig blijven, ongeacht nieuwe actuele data!

SCD-type 2 testen

Verander in sommige rijen uit het SDM een aantal waarden en run vervolgens de DWH-pijplijn.

Check vervolgens je **feit-** én **dimensietabellen**!

Piet is verhuisd van Leiden naar Leimuiden en heeft de dag erna een nieuwe bestelling geplaatst

Dimensie						Feit			
SK	Begintijd	Eindtijd	PK	Naam	Woonplaats	PK	Datum	Aantal	FK
1	2025-01-01	2026-01-02	1	Piet	Leiden	A	2025-01-01	100	1
2	2025-01-01	NULL	2	Elske	Leiden	B	2025-02-01	200	2
3	2025-01-01	NULL	3	Jan	Lisse	C	2025-03-01	300	3

Verwacht:

- Een ingevulde “Eindtijd”-kolom in de oude regel
- Een nieuwe regel met de actuele waarden

Verwacht:

Een nieuwe regel mét de nieuwe/hogste SK (4) als foreign key, i.p.v. de vorige SK (1).

Na vandaag...

Weet/ken je...

Kan je...

Welke soorten databronnen er bestaan

Wat Slowly Changing Dimensions typen 0 t/m 3 zijn, hoe zij van elkaar verschillen en waarom/wanneer je welk type gebruikt.

Wat in Datawarehouses het verschil is tussen Business Keys & Surrogate Keys

Welke inlaadstrategieën je het beste kunt toepassen bij het inladen van data in Datawarehouses

Slowly Changing Dimensions typen 1 en 2 inbouwen in dynamische Datawarehouses

Surrogate- & Business Keys op de juiste manier verwerken in Datawarehouses

De strategieën "UPSERT" en "Incremental Data Loading" gebruiken om Datawarehouses te actualiseren met data uit Source Data Modellen

HC + WC

